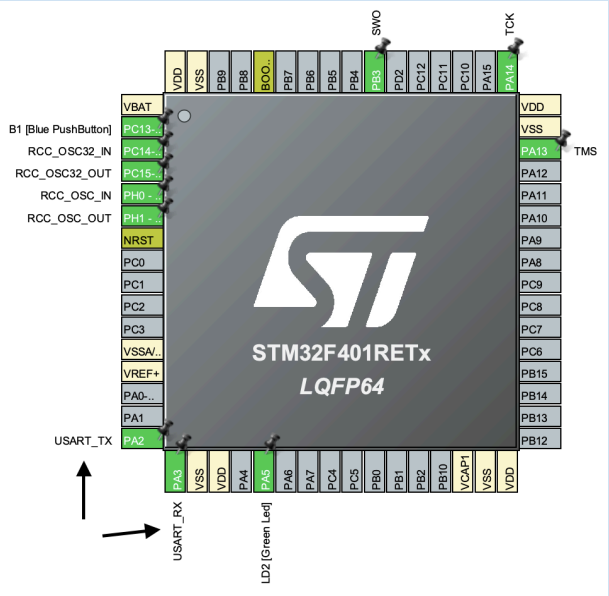
Mark	4/11

Team name:	B9		
Homework number:	Homework 04		
Due date:	14/10/2025		
Contribution:	No	Partial	Full
Pica Mirko			✅
Prendin Christian			✅
Pignataro Gabriele			✅
Moazenkakhki Mostafa			✅
Yarrabbotula Sunandha			✅
Notes:			

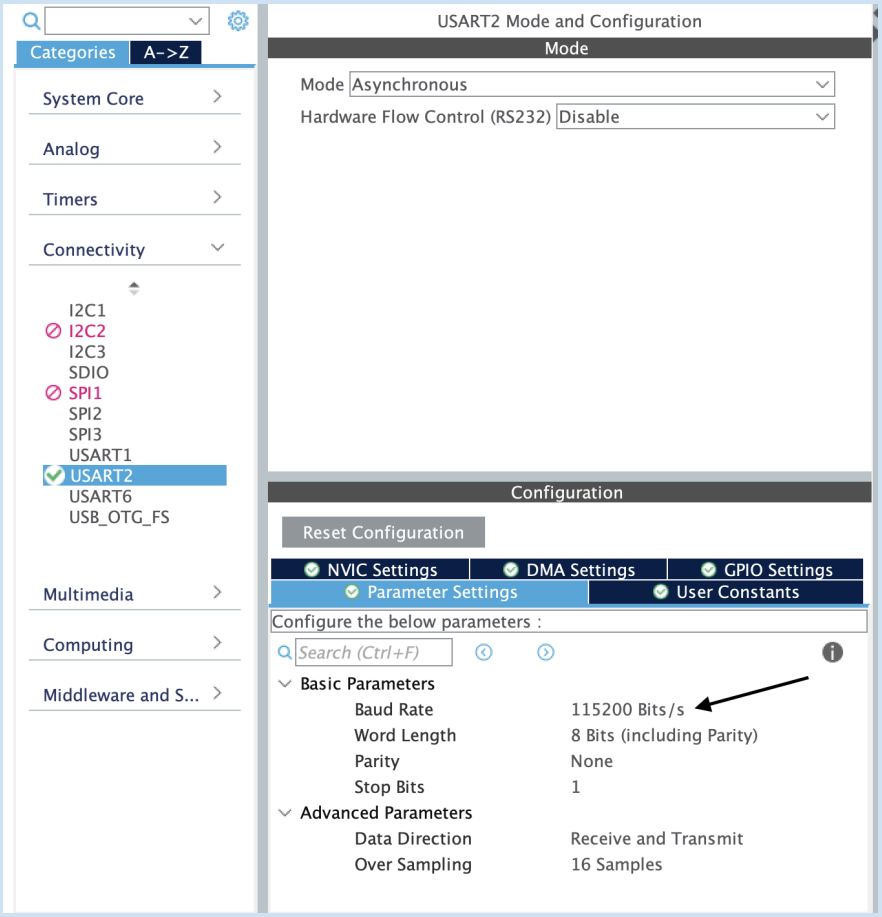
Project name	UART & LCD		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			✅

Part 1. Complete the UART project with DMA, as in slide 10 of pack 06.

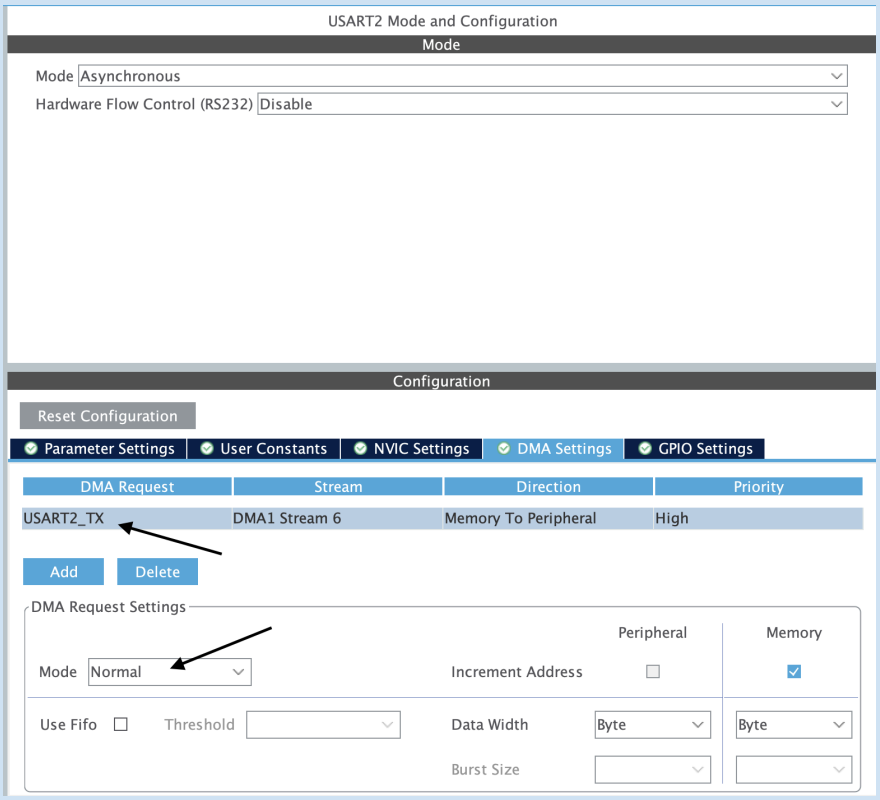
As written in the slides, we create the new project and check that the two pins for USART are enabled.



We select a baud rate (the same BR should be set in the terminal emulator, since it is an asynchronous communication).



We enable DMA in the USART2 peripheral with USART2_TX requests, in normal mode.



We enable its global interrupt.

NVIC Settings | DMA Settings | GPIO Settings
Parameter Settings | User Constants

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
DMA1 stream6 global interrupt	☑	0	0
USART2 global interrupt	☑	0	0

To send the string every second we configure TIM2 and its global interrupt to trigger every second.

TIM2 Mode and Configuration

Mode

Slave Mode: Disable
Trigger Source: Disable
Clock Source: Internal Clock
Channel1: Disable
Channel2: Disable
Channel3: Disable
Channel4: Disable
Combined Channels: Disable
☐ *Use ETR as Clearing Source*
☐ *XOR activation*
☐ One Pulse Mode

Configuration

Reset Configuration

User Constants | NVIC Settings | DMA Settings
Parameter Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings
- Prescaler (PSC - 16 bits val... 8399
- Counter Mode Up
- Counter Period (AutoReload... 9999
- Internal Clock Division (CKD) No Division
- auto-reload preload Disable

Trigger Output (TRGO) Parameters
- Master/Slave Mode (MSM bit) Disable (Trigger input effect not dela...
- Trigger Event Selection Reset (UG bit from TIMx_EGR)

User Constants | NVIC Settings | DMA Settings
Parameter Settings

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
TIM2 global interrupt	☑	0	0

Finally we go into the main.c and modify it as follows.

```c
/* USER CODE BEGIN PV */
uint8_t string[] = "Name Year\n";
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if (htim->Instance == TIM2){
        HAL_UART_Transmit_DMA(&huart2, string, sizeof(string)-1);
    }
}
/* USER CODE END 0 */
```

In the main we just started the TIM2.

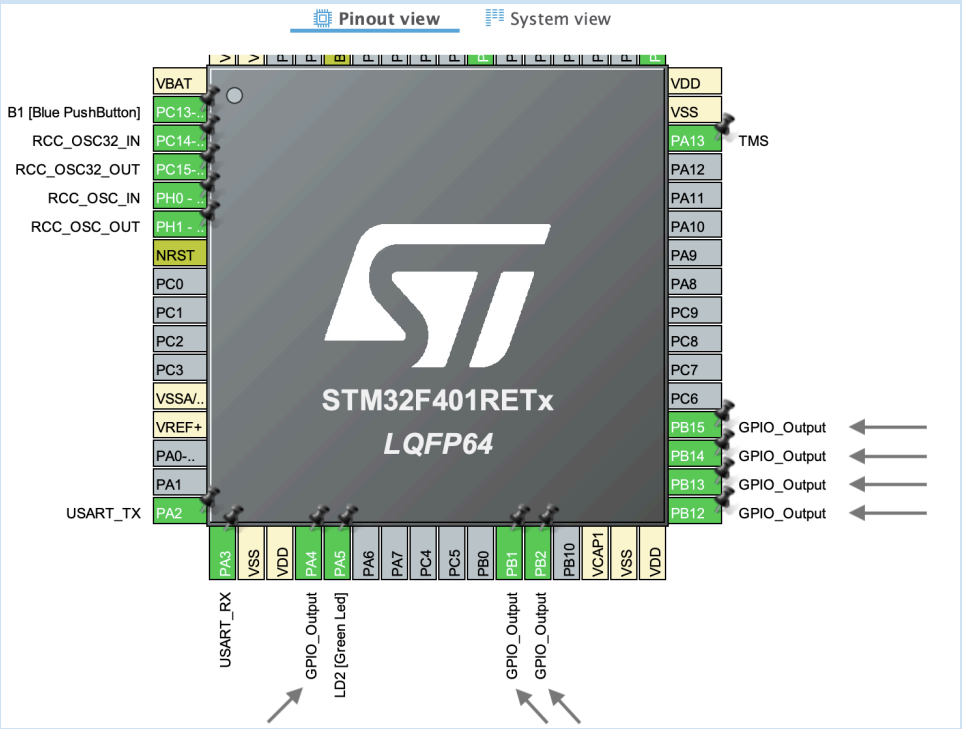
```c
  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_DMA_Init();
  MX_USART2_UART_Init();
  MX_TIM2_Init();
  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
```

We have tested it and it works.

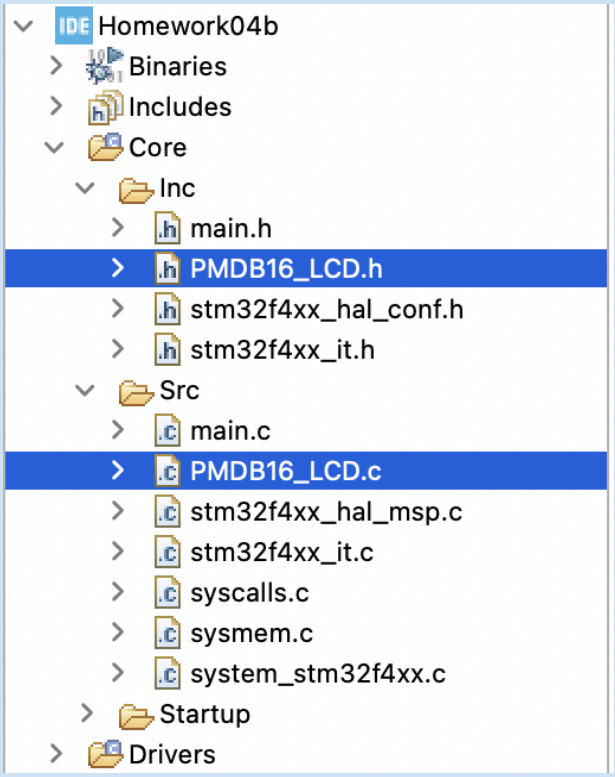
Part 2. Write on the LCD the name of each member of your group, one per line, in alphabetical order. Scroll every one second as indicated in the assignment.

We use the PMDB16 board's **parallel 4-bit** LCD interface to Nucleo-F401RE.
From the schematics:
- **LCD_RS → PB2**,
- **LCD_E → PB1**
- **LCD_D4..D7 → PB12..PB15**

Configure **PB1, PB2, PB12, PB13, PB14, PB15** as **GPIO_Output**, no pull, low speed.



As specified in the slides, we used the provided **PMDB16_LCD** library by importing the files "**PMDB16_LCD.c**" to the *project/Core/Source* folder and "**PMDB16_LCD.h**" to the *project/Core/Inc* folder, and in the end including it in our main.c



Then we could use its high-level APIs such as:
- lcd_initialize()
- lcd_println(char string[], uint8_t row)

We defined our names vector as following:

```
  /* USER CODE BEGIN 2 */
  lcd_initialize();
    int i = 0;
    char *names[] = {
        "Mirko",
        "Christian",
        "Gabriele",
        "Mostafa",
        "Sunandha"
    };
  /* USER CODE END 2 */
```

Then, in the while loop we handled the logic of printing names with a switch, and using the lcd_println function:

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
      switch (i){
          case 0:
              lcd_println(names[0], 1);
              i++;
              break;
          case 5:
              lcd_println(names[i-1], 0);
              lcd_println(names[0], 1);
              i = 1;
              break;
          default:
              lcd_println(names[i-1], 0);
              lcd_println(names[i], 1);
              i++;
          }
        HAL_Delay(1000);
  }
  /* USER CODE END 3 */
}
```

In the end, the program will enter the case 0 just in the first iteration of the while, for printing the first name in the bottom row and none in the upper one. Then, when the end is reached, we set i=1 so the first name will pop up in the bottom line after the last one in the upper one.

We have tested it and it works.

Professor comments:	